```
import crypto from 'node:crypto';
import { q, one } from '../db.js';

const sha256 = (s) => crypto.createHash('sha256').update(s).digest('hex');

// Every completed execution leaves a receipt. Receipts chain per business,
// so a removed or edited receipt breaks verification for everything after it.
export async function writeReceipt({ businessId, executionId, capability, verific
  const prev = await one(
    `select chain_hash from receipt where business_id = $1 order by created_at de
    [businessId]
  );
  const prevHash = prev?.chain_hash ?? null;

  const payload = JSON.stringify({ executionId, capability, verification, evidenc
  const payloadHash = sha256(payload);
  const chainHash = sha256((prevHash ?? '') + payloadHash);

  return one(
    `insert into receipt (execution_id, business_id, capability, verification, ev
     values ($1,$2,$3,$4,$5,$6,$7,$8) returning *`,
    [executionId, businessId, capability, verification, JSON.stringify(evidence),
  );
}

// Walk the chain forward. Returns the first receipt whose link does not hold.
export async function verifyChain(businessId) {
  const rows = await q(
    `select * from receipt where business_id = $1 order by created_at asc`, [busi
  );
  let prevHash = null;
  for (const r of rows) {
    const expected = sha256((prevHash ?? '') + r.payload_hash);
    if (r.prev_hash !== prevHash || r.chain_hash !== expected) {
      return { ok: false, brokenAt: r.id, count: rows.length };
    }
    prevHash = r.chain_hash;
  }
  return { ok: true, count: rows.length };
}
```